Technical Interview Master Notes

This PDF is designed for fast interview revision. It focuses on questions interviewers commonly ask, concise model answers, follow - up questions, and the reasoning behind the expected answer.

---

1. Data Structures & Algorithms (DSA)

Arrays & Strings

Q: What is an Array?

An array is a linear data structure that stores elements of the same data type in contiguous memory locations.

```
int[] arr = {1,2,3,4};
```

Interviewer expects: “ Array provides O(1) access using index because elements are stored continuously in memory. ”

Follow - up: Why is insertion in the middle costly?

Because elements need to be shifted.

Operation	Time Complexity
Access	O(1)
Search	O(n)
Insert	O(n)
Delete	O(n)

Q: Difference between Array and Linked List

Array	Linked List
Contiguous memory	Non - contiguous memory
Fast index access	Slow sequential access
Fixed size (classic arrays)	Dynamic size
O(1) index access	O(n) traversal

Sliding Window

Q: What is Sliding Window?

An optimization technique that maintains a moving window over contiguous elements so we avoid recomputing work from scratch.

Typical use: fixed - size subarrays, longest/shortest substring with constraints, running aggregates.

Example: Maximum sum subarray of size K.

```
Brute force: O(n*k)
Optimized sliding window: O(n)
```

Interviewer expects: " Use Sliding Window when the problem talks about contiguous subarrays/substrings and you can update the answer incrementally. "

Two Pointers

Q: When do we use Two Pointers?

Usually on sorted arrays/strings or when we need to shrink/expand a range from both ends.

Examples: Pair Sum, Remove Duplicates from Sorted Array, Container With Most Water.

Interviewer expects: " Two pointers often reduce $O(n^2)$ brute - force solutions to $O(n)$ by moving pointers based on a rule. "

Stack

Q: What is a Stack?

A LIFO (Last In First Out) data structure.

- Push
- Pop
- Peek

```
Push → O(1)
Pop  → O(1)
Peek → O(1)
```

Real - world example: Browser back button, undo operations.

Follow - up: Difference between Stack and Queue

Stack	Queue
LIFO	FIFO
Push/Pop	Enqueue/Dequeue

Queue

Q: What is a Queue?

A FIFO (First In First Out) data structure.

Real - world examples: Ticket booking systems, printer queues.

Linked List

Q: Why use a Linked List?

Dynamic size and efficient insert/delete operations when you already have the node reference.

Types: Singly, Doubly, Circular.

Q: How do you detect a cycle?

Use Floyd ' s Cycle Detection (Tortoise and Hare): slow moves 1 step, fast moves 2 steps. If they meet, a cycle exists.

```
O(n) time, O(1) extra space
```

Trees

Q: What is a Tree?

A hierarchical data structure consisting of nodes connected by edges.

Common terms: Root, Parent, Child, Leaf.

Traversals

Traversal	Order
Inorder	Left → Root → Right
Preorder	Root → Left → Right
Postorder	Left → Right → Root

Follow - up: Why does inorder traversal of a BST produce sorted output?

Because a BST maintains Left < Root < Right for every node.

Binary Search Tree (BST)

Q: What is a BST?

A binary tree where left subtree values are smaller than the root and right subtree values are larger.

```
Left Child  < Root
Right Child > Root
```

Case	Search Complexity
Average (balanced)	O(log n)
Worst (skewed)	O(n)

Graphs

Q: What is a Graph?

A collection of vertices (nodes) and edges.

Examples: Maps, social networks, dependency graphs.

BFS vs DFS

BFS	DFS
Uses a Queue	Uses a Stack/Recursion
Level - wise traversal	Depth - wise traversal
Finds shortest path in unweighted graphs	Does not guarantee shortest path

Interviewer expects: “ Use BFS for shortest path in an unweighted graph; use DFS for reachability, cycle detection, and exhaustive exploration. ”

Dynamic Programming (DP)

Q: What is Dynamic Programming?

An optimization technique where we solve overlapping subproblems once and reuse the stored results.

Key approaches

Approach	Idea
Memoization	Top - down recursion + cache
Tabulation	Bottom - up iterative DP table

Starter problems: Fibonacci, 0/1 Knapsack basics, House Robber, Longest Common Subsequence.

Interview tip: Explain the state, transition, base cases, and complexity.

2. Core CS Fundamentals

Object - Oriented Programming (OOP)

Encapsulation: Wrapping data and methods together; control access with private fields and public methods.

```
class Employee {
    private int salary;
}
```

Inheritance: A child class acquires properties/behavior of a parent class.

Polymorphism: One interface, multiple implementations (compile - time overloading, run - time overriding).

Abstraction: Expose essential behavior and hide implementation details (ATM, car controls, etc.).

DBMS

Q: What is Normalization?

Reducing redundancy and improving consistency by organizing data into normal forms.

Important forms: 1NF, 2NF, 3NF.

Q: What is a Primary Key?

A column (or set of columns) that uniquely identifies each row.

Q: What is a Foreign Key?

A column that references a key in another table to enforce relationships.

DELETE vs TRUNCATE vs DROP

DELETE	TRUNCATE	DROP
Removes selected rows	Removes all rows quickly	Removes the table/schema object
Can be filtered with WHERE	No WHERE clause	Object no longer exists
Row - by - row logging (DB - dependent)	Minimal logging (DB - dependent)	Metadata operation (DB - dependent)

SQL topics interviewers frequently ask:

- INNER JOIN
- LEFT JOIN
- GROUP BY
- HAVING
- INDEX basics

Operating Systems

Process vs Thread

Process	Thread
Independent execution unit	Lightweight execution unit within a process
Own address space	Shares process memory
Higher context - switch overhead	Lower overhead

Q: What is Deadlock?

A situation where processes wait indefinitely for resources held by each other.

Necessary conditions: Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait.

Q: What is Context Switching?

The CPU saves one process/thread state and restores another, allowing multitasking.

Computer Networks

HTTP vs HTTPS

HTTP	HTTPS
No encryption	Encrypted with TLS/SSL
Port 80 (typical)	Port 443 (typical)
Susceptible to interception	Protects confidentiality and integrity

Q: What is TCP?

A reliable, connection - oriented protocol with acknowledgements, retransmissions, flow control, and ordered delivery.

Q: What is a REST API?

A stateless HTTP - based API style that models resources and uses methods such as GET, POST, PUT, and DELETE.

3. Problem - Solving Strategy

- Understand the problem and constraints before coding.
- Do a small dry run with an example input.
- Start with a brute - force solution, then optimize.
- Write clean step - by - step logic before implementation.
- Practice consistently (1 – 2 problems daily is enough if sustained).
- Revise old problems weekly to reinforce patterns.

Interview flow that sounds strong:

- Clarify constraints and edge cases.
- State brute - force complexity.
- Identify the pattern (hashing, sliding window, BFS, DP, etc.).
- Propose the optimized approach.
- Discuss time/space complexity before coding.

4. Important Interview Questions (Short, Interview - Ready Answers)

Q: What is time complexity and space complexity?

A: Time complexity measures how running time grows with input size; space complexity measures additional memory used. We usually express both with Big - O notation.

Q: Difference between stack and queue?

A: Stack is LIFO; queue is FIFO. Stack uses push/pop; queue uses enqueue/dequeue.

Q: What is hashing and where is it used?

A: Hashing maps keys to buckets/indices for fast average - case lookup. Used in hash maps, hash sets, caches, indexing, and duplicate detection.

Q: What is BFS vs DFS?

A: BFS explores level by level using a queue and finds shortest paths in unweighted graphs. DFS explores depth first using recursion/stack and is useful for reachability, cycle detection, and exhaustive search.

Q: What is normalization in DBMS?

A: Organizing data to reduce redundancy and improve consistency by splitting tables based on dependencies.

Q: Explain deadlock conditions.

A: Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait are the four necessary conditions for deadlock.

Q: What is a REST API?

A: A stateless HTTP - based API style that exposes resources and operations using methods like GET, POST, PUT, and DELETE.

5. HR Interview Answers (Professional, Concise)

Tell me about yourself

A strong structure is: Education → Experience → Current Skills → Why this role.

Sample: “ Hi, I ’ m Sri Gavi. I ’ m a Software Engineer with experience in Java, Spring Boot, SQL, and microservices. I enjoy solving backend problems and have recently focused on DSA and system design preparation. I ’ m looking for opportunities where I can contribute while continuing to grow as an engineer. ”

Why should we hire you?

“ I have strong problem - solving skills, hands - on development experience, and a willingness to learn quickly. I focus on delivering maintainable solutions and communicating clearly with the team. ”

What are your strengths?

- Fast learner
- Problem - solving mindset
- Team collaboration
- Ownership and accountability

What is your weakness?

“ Earlier I used to spend too much time perfecting solutions. I ’ ve learned to balance quality with delivery timelines by time - boxing investigation and iterating. ”

Why do you want to join our company?

Formula: Company + Learning + Contribution.

“ I ’ m interested in your engineering culture and the scale of problems you work on. The role aligns with my backend and problem - solving strengths, and I believe I can contribute while learning from a strong engineering team. ”

6. Final Advice

- Do not memorize solutions—learn patterns and trade - offs.
- Say the complexity out loud before coding.
- Think in terms of constraints and invariants.
- Consistency beats intensity for interview prep.
- For senior/backend roles, add System Design: load balancers, caches, database scaling, microservices, messaging systems (Kafka), and CAP theorem basics.